Sorting II
Merge Sort, Quick Sort

Data Structures & Object-Oriented Programming



Table of Contents

- Recursion Tree
- Merge Sort
- Quick Sort
- Python Timsort
- Code ของทั้ง 2 อัลกอริทึม

บทนำ

- สัปดาห์ที่แล้วเราได้เรียนรู้ sorting algorithms สามตัวแรก (Bubble, Selection, Insertion Sort)
  - มีความซับซ้อนแบบ $O(n^2)$
  - ความซับซ้อนระดับนี้ช้าเกินไปสำหรับข้อมูลขนาดใหญ่มาก (ต่างจาก O(n log n) ถึง 50,000 เท่าเมื่อ n มีขนาด 1M รายการ)
- สัปดาห์นี้ เรียน sorting algorithms กลุ่มใหม่ ได้แก่ Merge Sort และ Quick Sort
  - ทั้งคู่ใช้แนวคิดการออกแบบ algorithm ที่เรียกว่า Divide and Conquer

ปัญหาของ $O(n^2)$ ในโลกจริง

- Bubble Sort, Selection Sort และ Insertion Sort มีความซับซ้อนแบบ $O(n^2)$
  - บนข้อมูลขนาด n = 1M รายการ จำเป็นต้องทำ operation มากถึง 10^{12} ครั้ง คิดเป็นเวลาประมาณ 17 นาที
- ตัวเลขนี้อาจยอมรับได้ในบางบริบท แต่ในสถานการณ์ที่ระบบต้องจัดเรียงข้อมูลจำนวนมากแบบต่อเนื่องและรวดเร็ว เช่น
  - search engine ที่ต้องจัดเรียงผลการค้นหาหลายล้านรายการให้เสร็จภายในเสี้ยววินาที
  - ระบบฐานข้อมูลที่ต้อง sort ผลลัพธ์ของ query แบบ real-time ให้ผู้ใช้งานหลายพันคนพร้อมกัน
  - ในบริบทเหล่านี้ ความล่าช้าระดับนาทีถือเป็นสิ่งที่ยอมรับไม่ได้โดยสิ้นเชิง
- คำถามที่ตามมาคือ เป็นไปได้หรือไม่ที่จะออกแบบ sorting algorithm ให้มีความซับซ้อนต่ำกว่า $O(n^2)$
  - เป็นไปได้ ด้วยแนวคิดการออกแบบ algorithm ที่เรียกว่า Divide and Conquer

Divide and Conquer คืออะไร

- เป็น paradigm การออกแบบ algorithm ที่ประกอบด้วยสามขั้นตอนหลักเสมอ
  - Divide (แบ่ง) คือ การแบ่งปัญหาขนาดใหญ่ออกเป็นปัญหาย่อยที่มีโครงสร้างเหมือนกันทุกประการกับปัญหาเดิม เพียงแต่มีขนาดเล็กลง
  - Conquer (พิชิต) คือ การแก้ปัญหาย่อยแต่ละชิ้นด้วยกลไก recursion (การเรียกตัวเองซ้ำ) จนกระทั่งปัญหาย่อยมีขนาดเล็กพอที่จะแก้ได้โดยตรงในทันที (เรียกว่า base case)
  - Combine (รวม) คือ การนำผลลัพธ์ของปัญหาย่อยทั้งหมดมารวมเข้าด้วยกัน เพื่อประกอบเป็นคำตอบของปัญหาใหญ่ดั้งเดิม

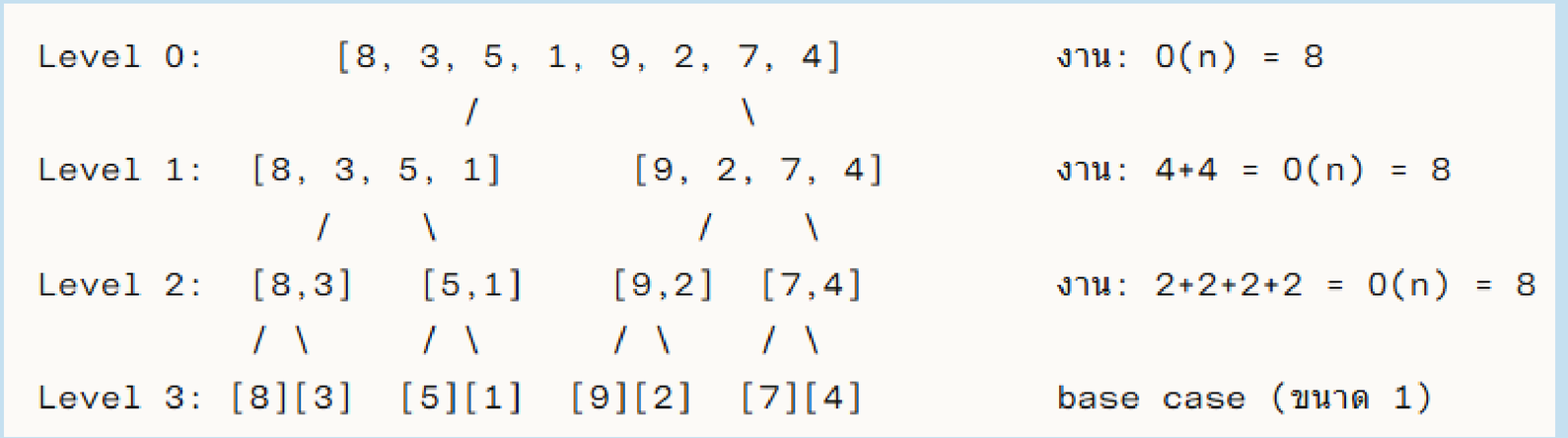
Divide and Conquer คืออะไร (2)

- แนวคิด Divide and Conquer ทำให้ลด “ปริมาณงาน” ที่ต้องจัดการในแต่ละครั้งลงอย่างรวดเร็วมาก
  - เมื่อแบ่งข้อมูลออกเป็นครึ่งหนึ่งซ้ำไปเรื่อยๆ ขนาดของปัญหาจะลดจาก n ลงมาเหลือ 1 ภายในเพียง **$\log_2(n)$** ขั้นตอนเท่านั้น ซึ่งเป็นตัวเลขที่เติบโตช้ามากเมื่อเทียบกับ n
  - ตัวอย่างเช่น หาก n = 1,000,000 ค่า $\log_2(n)$ มีค่าประมาณ 20 เท่านั้น (เทียบกับ n เองที่มีค่า 1M)
  - ความแตกต่างของอัตราการเติบโตนี้เองที่เป็นกุญแจสำคัญที่ทำให้ Merge Sort และ Quick Sort มีความซับซ้อนแบบ **O(n log n)** ซึ่งเร็วกว่า $O(n^2)$ อย่างมีนัยสำคัญ

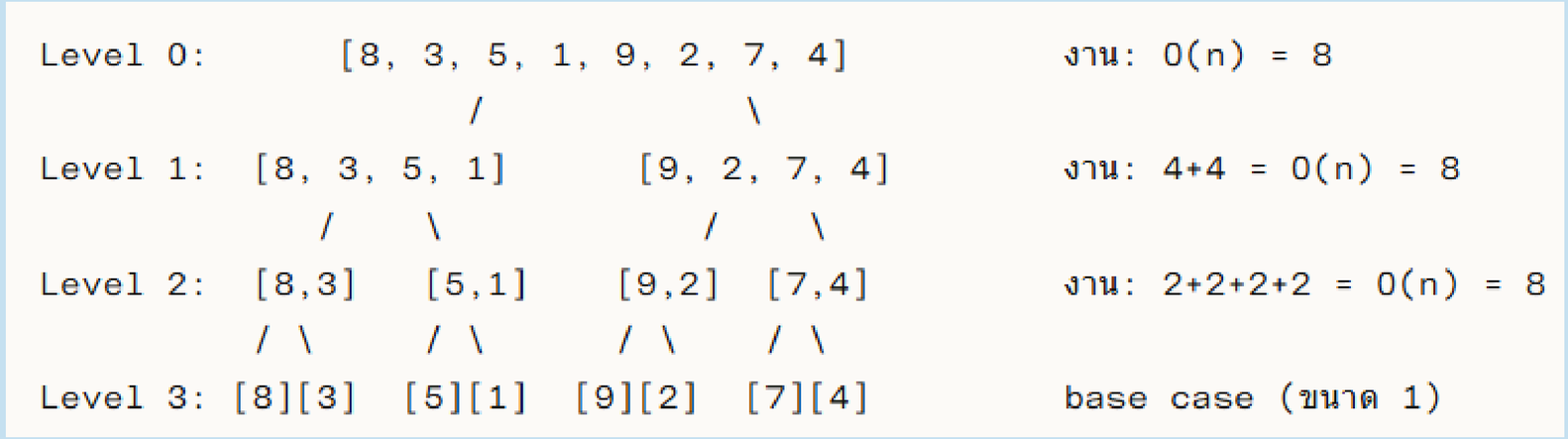
Recursion Tree

โครงสร้างของ Recursion Tree

- Recursion Tree เป็นเครื่องมือหลักที่ใช้วิเคราะห์ความซับซ้อนของ algorithm แบบ Divide and Conquer
- Merge Sort ที่ทำงานบน array ขนาด n = 8 เช่น [8, 3, 5, 1, 9, 2, 7, 4]
  - โครงสร้างของการเรียก recursion สามารถวาดเป็นแผนภาพต้นไม้ได้ดังนี้

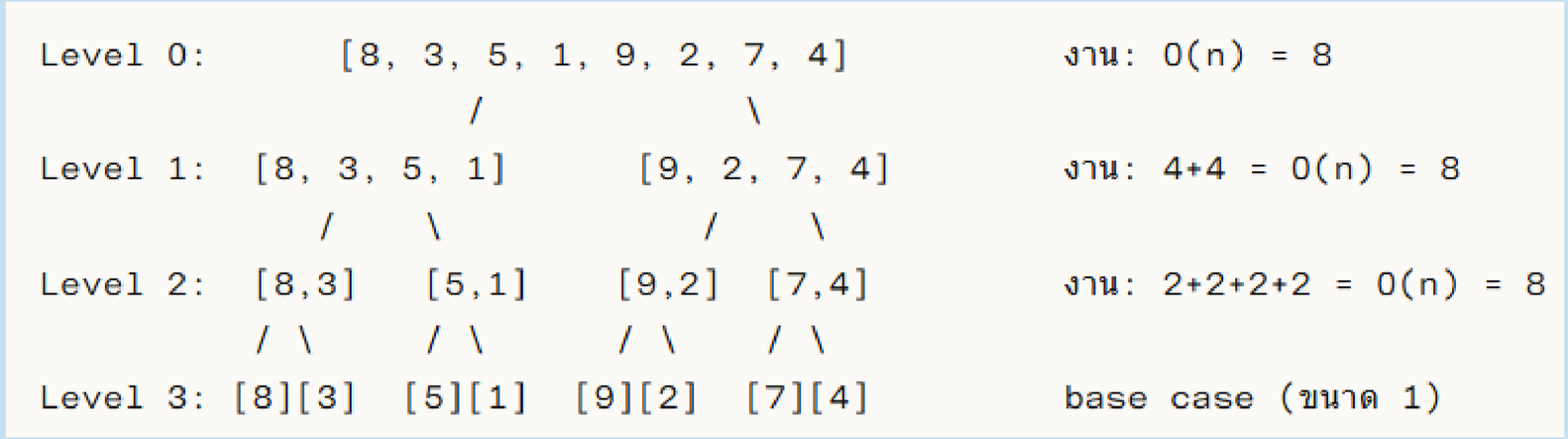


โครงสร้างของ Recursion Tree



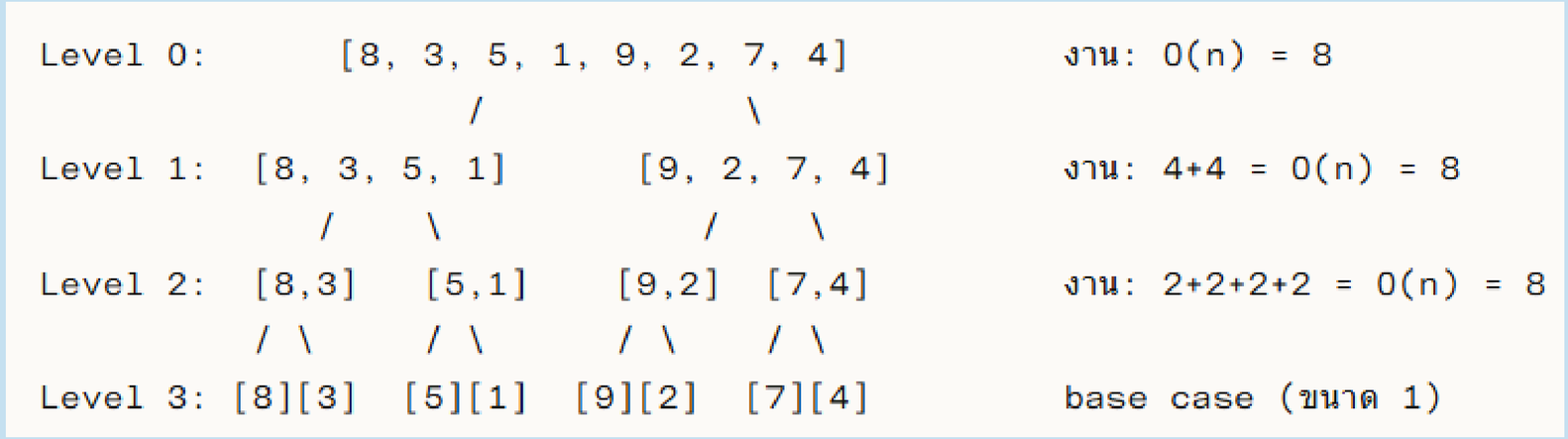
- จำนวน level ทั้งหมดคือ $\log_2(n)$
  - ในแต่ละ level ขนาดของ sub-array จะถูกแบ่งครึ่งลงเรื่อยๆ จาก n ไปเป็น n/2, n/4, n/8, … จนกระทั่งเหลือ 1
  - คำถามคือต้องแบ่งครึ่งกี่ครั้งจึงจะได้ขนาด 1 คำตอบคือ $\log_2(n)$ ครั้งพอดี
  - ตัวอย่างที่ n=8 จะได้ $\log_2(8) = 3$ ซึ่งตรงกับที่เห็นในแผนภาพว่ามี Level 1, 2, 3 อยู่ใต้ Level 0 รวมเป็น 3 ระดับของการแบ่ง (หรือกล่าวได้ว่ามีทั้งหมด 4 level นับรวม Level 0)

โครงสร้างของ Recursion Tree



- งานรวมในแต่ละ level เป็น O(n) เสมอ ไม่ว่าจะเป็น level ใดก็ตาม
  - แม้ว่าในแต่ละ level จะมีจำนวน sub-array มากขึ้นเรื่อยๆ (1, 2, 4, 8, …) แต่ขนาดของแต่ละ sub-array ก็เล็กลงตามสัดส่วนที่เท่ากันพอดี (8, 4, 2, 1)
  - เมื่อคูณจำนวน sub-array ด้วยขนาดของแต่ละตัวในทุก level จะได้ผลลัพธ์คงที่เท่ากับ n เสมอ
  - เหตุผลคือ ไม่ว่าข้อมูลจะถูกแบ่งเป็นกี่ส่วน element ทั้งหมดจากต้นฉบับยังคงถูกนับรวมและประมวลผลครบทุกตัวในทุก level เสมอ เพียงแต่กระจัดกระจายอยู่คนละ sub-array เท่านั้น

โครงสร้างของ Recursion Tree



- เมื่อรวมสองส่วนนี้เข้าด้วยกัน จะได้ความซับซ้อนโดยรวมเท่ากับ
  - (งานต่อ level) × (จำนวน level) = O(n) × O(log n) = O(n log n)
  - ซึ่งเป็นที่มาของความซับซ้อนที่เห็นในทั้ง Merge Sort และ Quick Sort (ในกรณีที่การแบ่งมีความสมดุลดี)

Merge Sort

หลักการทำงานของ Merge Sort

มีสามขั้นตอนดังนี้

- Divide: แบ่ง array ออกเป็นสองครึ่งที่จุดกึ่งกลาง
- Conquer: เรียงลำดับแต่ละครึ่งด้วย recursion (เรียก Merge Sort ซ้ำกับแต่ละครึ่ง) จนกระทั่งถึง base case
- Combine: รวมสองครึ่งที่เรียงลำดับเสร็จแล้วกลับเข้าด้วยกัน ผ่านขั้นตอนที่เรียกว่า merge
- ขั้นตอน merge คือ หัวใจสำคัญที่สุดของทั้ง algorithm เพราะเป็นจุดเดียวที่มีการทำงานจริงเกิดขึ้น
    - ขั้นตอน divide เป็นเพียงการคำนวณตำแหน่งกึ่งกลาง ไม่มีการประมวลผลข้อมูลใดๆ

วิธีการ Merge สอง Sorted Array เข้าด้วยกัน

- สมมติว่ามี array ที่เรียงลำดับแล้วสองชุดคือ left = [1, 3, 5, 8] และ right = [2, 4, 7, 9] และต้องการรวมทั้งสองเข้าเป็น array เดียวที่ยังคงเรียงลำดับอยู่
- วิธีการทำงานคือใช้ตัวชี้ตำแหน่งสองตัว (สมมติ i สำหรับ left และ j สำหรับ right)
- เปรียบเทียบค่าที่ตัวชี้ทั้งสองชี้อยู่ แล้วนำค่าที่น้อยกว่าใส่ลงใน array ผลลัพธ์ก่อน จากนั้นเลื่อนตัวชี้ของฝั่งนั้นไปข้างหน้าหนึ่งตำแหน่ง ทำซ้ำไปเรื่อยๆ จนกว่าฝั่งใดฝั่งหนึ่งจะหมด

```
left  = [1, 3, 5, 8]
right = [2, 4, 7, 9]

1 < 2  → append 1, เลื่อน i        result = [1]
3 > 2  → append 2, เลื่อน j        result = [1, 2]
3 < 4  → append 3, เลื่อน i        result = [1, 2, 3]
5 > 4  → append 4, เลื่อน j        result = [1, 2, 3, 4]
5 < 7  → append 5, เลื่อน i        result = [1, 2, 3, 4, 5]
8 > 7  → append 7, เลื่อน j        result = [1, 2, 3, 4, 5, 7]

left เหลือ [8] (right หมดแล้ว) → นำส่วนที่เหลือทั้งหมดต่อท้ายทันที
result = [1, 2, 3, 4, 5, 7, 8, 9]
```

หลักการทำงานของ Merge Sort

- เมื่อฝั่งใดฝั่งหนึ่งหมดก่อน (ในตัวอย่างนี้คือ right หมดก่อนเพราะเทียบ 8 กับ 7 แล้ว 7 ถูกนำออกไปหมดแล้ว)
- ส่วนที่เหลือของอีกฝั่งหนึ่งสามารถนำมาต่อท้ายผลลัพธ์ได้ทันทีโดยไม่ต้องเปรียบเทียบอีก
  - ทราบอยู่แล้วว่าฝั่งที่เหลือนั้นเรียงลำดับอยู่แล้วภายในตัวเอง และมีค่ามากกว่าทุกตัวที่ถูกนำเข้า result ไปแล้วทั้งหมด
- ความซับซ้อนของขั้นตอน merge นี้คือ O(n) โดยที่ n คือผลรวมของความยาวทั้งสอง array
  - เพราะแต่ละ element จากทั้งสองฝั่งจะถูกนำเข้า result เพียงครั้งเดียวเท่านั้น ไม่มีการเปรียบเทียบซ้ำหรือย้อนกลับใดๆ

```
left  = [1, 3, 5, 8]
right = [2, 4, 7, 9]

1 < 2  → append 1, เลื่อน i        result = [1]
3 > 2  → append 2, เลื่อน j        result = [1, 2]
3 < 4  → append 3, เลื่อน i        result = [1, 2, 3]
5 > 4  → append 4, เลื่อน j        result = [1, 2, 3, 4]
5 < 7  → append 5, เลื่อน i        result = [1, 2, 3, 4, 5]
8 > 7  → append 7, เลื่อน j        result = [1, 2, 3, 4, 5, 7]

left เหลือ [8] (right หมดแล้ว) → นำส่วนที่เหลือทั้งหมดต่อท้ายทันที
result = [1, 2, 3, 4, 5, 7, 8, 9]
```

Merge Sort เป็น O(n log n) ทุกกรณี ไม่มีข้อยกเว้น

- Merge Sort คือมีความซับซ้อนเป็น O(n log n) ในทุกกรณี ไม่ว่าจะเป็น best, average หรือ worst case ก็ตาม
  - เพราะโครงสร้างของ recursion tree ของ Merge Sort ไม่เปลี่ยนแปลงไปตามลักษณะของ input ไม่ว่า array ที่รับเข้ามาจะเรียงลำดับไว้แล้ว เรียงกลับหัว หรือสุ่มแบบใดก็ตาม
  - ขั้นตอน divide จะแบ่งครึ่งที่ตำแหน่งกึ่งกลางเสมอ (ตาม index ไม่ได้ขึ้นกับค่าของข้อมูล) ทำให้จำนวน level และปริมาณงานต่อ level คงที่เสมอ
- คุณสมบัตินี้แตกต่างจาก Quick Sort ซึ่งโครงสร้างการแบ่งขึ้นอยู่กับค่าของข้อมูลโดยตรง

Merge Sort เป็น Stable Sort

- กลไกที่ทำให้เกิดคุณสมบัตินี้อยู่ที่เงื่อนไขการเปรียบเทียบภายในขั้นตอน merge ซึ่งต้องใช้ <= (น้อยกว่าหรือเท่ากับ) แทนที่จะเป็น < (น้อยกว่าอย่างเคร่งครัด)
- เมื่อค่าจาก left และ right เท่ากันพอดี เงื่อนไข left[i] <= right[j] จะเป็นจริง ทำให้ค่าจาก left ถูกนำเข้า result ก่อน เสมอ
  - เนื่องจาก left มาจากฝั่งซ้ายของ array ต้นฉบับ (ซึ่งควรมาก่อนตามลำดับเดิมอยู่แล้ว)
- การจัดลำดับความสำคัญให้ left มาก่อนในกรณีที่เท่ากันนี้เองที่รักษาลำดับสัมพัทธ์เดิมของ element ที่มีค่าเท่ากันไว้ได้อย่างถูกต้อง

ข้อเสียของ Merge Sort

- ขั้นตอน merge จำเป็นต้องสร้าง array ใหม่ (result) ขึ้นมาเก็บผลลัพธ์ชั่วคราวในทุกครั้งที่มีการเรียก
- ทำให้ Merge Sort ใช้หน่วยความจำเพิ่มเติมเป็น O(n)
- ต่างจาก sorting algorithm แบบ in-place ที่ทำงานบน array เดิมโดยตรงโดยไม่ต้องสร้าง array ใหม่เลย

Quick Sort

หลักการทำงานของ Quick Sort

- Quick Sort ใช้แนวคิด Divide and Conquer แต่มีวิธีการแบ่งปัญหาที่แตกต่างกันโดยสิ้นเชิง
- Quick Sort จะเลือก element หนึ่งตัวขึ้นมาเรียกว่า pivot แล้วจัดเรียง array ใหม่ (เรียกกระบวนการนี้ว่า partition) ให้
  - element ที่มีค่าน้อยกว่าหรือเท่ากับ pivot อยู่ทางซ้ายทั้งหมด
  - element ที่มีค่ามากกว่า pivot อยู่ทางขวาทั้งหมด
  - จากนั้นเรียก recursion บนทั้งสองฝั่งแยกกัน
- หลังจากขั้นตอน partition เสร็จสิ้น pivot จะอยู่ในตำแหน่งสุดท้ายที่ถูกต้องแล้วทันที ไม่ต้องมีการเคลื่อนย้ายอีกตลอดการทำงานที่เหลือ

Lomuto Partition Scheme

- วิธีการ partition ที่นิยมใช้กันมากที่สุดวิธีหนึ่งเรียกว่า Lomuto Partition Scheme
  - ใช้ element ตัวสุดท้ายของช่วงที่กำลังพิจารณาเป็น pivot เสมอ
- ลองพิจารณาการ trace บน arr = [3, 6, 8, 10, 1, 2, 1]
  - โดยกำหนด low=0, high=6 (pivot คือ arr[high] = arr[6] = 1)
  - หลักการทำงานใช้ตัวชี้สองตัว คือ
    - i ทำหน้าที่ชี้ตำแหน่งสุดท้ายของ “โซนที่มีค่าน้อยกว่าหรือเท่ากับ pivot” (เริ่มต้นที่ low - 1 เพราะยังไม่มี element ใดอยู่ในโซนนี้เลย)
    - j ทำหน้าที่ไล่สำรวจ array ตั้งแต่ low ถึง high - 1

Lomuto Partition Scheme

- arr = [3, 6, 8, 10, 1, 2, 1]
- i ทำหน้าที่ชี้ตำแหน่งสุดท้ายของ “โซนที่มีค่าน้อยกว่าหรือเท่ากับ pivot” (เริ่มต้นที่ low - 1 เพราะยังไม่มี element ใดอยู่ในโซนนี้เลย)
- j ทำหน้าที่ไล่สำรวจ array ตั้งแต่ low ถึง high - 1

```
pivot = arr[6] = 1,  i = -1

j=0: arr[0]=3 > 1  → ข้าม (ไม่ทำอะไร)
j=1: arr[1]=6 > 1  → ข้าม
j=2: arr[2]=8 > 1  → ข้าม
j=3: arr[3]=10 > 1 → ข้าม
j=4: arr[4]=1 <= 1 → i++ (i=0), swap(0,4) → [1, 6, 8, 10, 3, 2, 1]
j=5: arr[5]=2 > 1  → ข้าม

สิ้นสุด loop: swap(i+1=1, high=6) → [1, 1, 8, 10, 3, 2, 6]
pivot_index = 1
```

- element ก่อนตำแหน่ง 1 (คือ arr[0]=1) มีค่าน้อยกว่าหรือเท่ากับ pivot (1) และ element หลังตำแหน่ง 1 ทั้งหมด (8, 10, 3, 2, 6) มีค่ามากกว่า pivot ทั้งหมด ✓
- เป้าหมายที่แท้จริงของขั้นตอน partition ไม่ใช่การเรียงลำดับให้เสร็จสมบูรณ์ในครั้งเดียว แต่เป็นการรับประกันว่า pivot อยู่ในตำแหน่งที่ถูกต้องแล้ว โดยมีค่าที่เล็กกว่าอยู่ซ้ายและใหญ่กว่าอยู่ขวาทั้งหมด

กลยุทธ์การเลือก Pivot และผลกระทบต่อ Complexity

- การเลือกว่าจะใช้ element ใดเป็น pivot มีผลกระทบอย่างมหาศาลต่อประสิทธิภาพของ Quick Sort
  - pivot ที่ดีควรแบ่ง array ออกเป็นสองส่วนที่มีขนาดใกล้เคียงกัน เพื่อให้ recursion tree มีความสมดุล (คล้ายกับ Merge Sort)
  - หาก pivot ที่เลือกมีค่าน้อยที่สุดหรือมากที่สุดใน array อยู่เสมอ การแบ่งจะกลายเป็น “หนึ่งฝั่งมี 0 หรือ 1 element ส่วนอีกฝั่งมีเกือบทั้งหมด” ซึ่งทำให้ recursion tree มีความลึกมากผิดปกติ

กลยุทธ์การเลือก Pivot และผลกระทบต่อ Complexity

- **กลยุทธ์ที่ 1 — Last-element pivot**
  - ใช้ element ตัวสุดท้ายของช่วงเป็น pivot เสมอ (แบบตัวอย่าง)
  - วิธีนี้เขียนง่ายที่สุด แต่มีปัญหาร้ายแรงเมื่อ input เรียงลำดับไว้แล้ว
  - ลองพิจารณา [1, 2, 3, 4, 5]

```
pivot = 5 (ตัวสุดท้าย)     → partition: [1,2,3,4] | [5]
pivot = 4                  → partition: [1,2,3]   | [4,5]
pivot = 3                  → partition: [1,2]     | [3,4,5]
...
```

- ในทุกครั้งที่ partition เกิดขึ้น ฝั่งขวามีเพียง pivot ตัวเดียว ในขณะที่ฝั่งซ้ายมี element ที่เหลือเกือบทั้งหมด
- ทำให้ tree depth เท่ากับ n พอดี แทนที่จะเป็น $\log_2(n)$
  - ความซับซ้อนกลายเป็น $O(n^2)$ ในกรณีนี้ ซึ่งเป็น worst case ของ Quick Sort

กลยุทธ์การเลือก Pivot และผลกระทบต่อ Complexity

- **กลยุทธ์ที่ 2 — Random pivot**
  - เลือก element แบบสุ่มขึ้นมาเป็น pivot ในแต่ละครั้ง
  - วิธีนี้ทำให้โอกาสที่จะเจอ worst case ต่ำมาก (เพราะ pattern ของ input ที่ทำให้แย่ที่สุดไม่ได้ถูกกำหนดตายตัวอีกต่อไป) แต่ไม่มีการรับประกันว่าจะไม่เจอ worst case เลย เพียงแต่มีความน่าจะเป็นต่ำมากเท่านั้น
  - average case จะได้ O(n log n)

กลยุทธ์การเลือก Pivot และผลกระทบต่อ Complexity

- **กลยุทธ์ที่ 3 — Median-of-three**
  - นำ element สามตัว ที่ตำแหน่ง low, mid (จุดกึ่งกลาง), และ high มาหาค่ามัธยฐาน (median) แล้วใช้ค่านั้นเป็น pivot
  - ลองพิจารณา [1, 2, 3, 4, 5]

```
arr[low]=1, arr[mid]=3, arr[high]=5 → median = 3
partition รอบ pivot=3: [1,2] | [3] | [4,5]   ← สมดุลกว่ามาก
```

- ด้วย median-of-three การแบ่งครั้งนี้ได้สัดส่วนประมาณครึ่งต่อครึ่งพอดี (2 ต่อ 2)
- ต่างจาก last-element pivot ที่ได้ 4 ต่อ 0 อย่างสิ้นเชิง
- กลยุทธ์นี้เป็นมาตรฐานที่นิยมใช้ใน production code เพราะสามารถหลีกเลี่ยง worst case บนทั้ง sorted และ reverse-sorted input ได้อย่างมีประสิทธิภาพ โดยไม่ต้องพึ่งพาความน่าจะเป็นเหมือน random pivot

คุณสมบัติสำคัญของ Quick Sort

- Quick Sort เป็น in-place sort ที่ทำงานบน array เดิมโดยตรง ใช้หน่วยความจำเพิ่มเติมเพียง O(log n) เท่านั้น (สำหรับเก็บ call stack ของการเรียก recursion)
  - น้อยกว่า Merge Sort ที่ต้องการ O(n) อย่างมาก
- แม้ worst case ของ Quick Sort จะเป็น $O(n^2)$ แต่ในทางปฏิบัติ Quick Sort มักทำงานได้เร็วกว่า Merge Sort เพราะมี cache performance ที่ดีกว่า
  - ทำงานบน array เดิมต่อเนื่องกันในหน่วยความจำ ไม่ต้องสร้าง array ใหม่เหมือน Merge Sort
- แต่ Quick Sort ไม่ใช่ stable sort
  - ขั้นตอน partition อาศัยการ swap element ที่อาจอยู่ห่างกันมากในการจัดตำแหน่ง
  - การ swap เช่นนี้สามารถทำให้ element ที่มีค่าเท่ากันสองตัวสลับลำดับสัมพัทธ์กันได้

Python Timsort

Python Timsort

- ฟังก์ชัน sorted() และ method .sort() ที่มีอยู่ใน Python ไม่ได้ใช้ Merge Sort หรือ Quick Sort แบบล้วนๆ แต่ใช้ algorithm ที่เรียกว่า Timsort
  - เป็น hybrid algorithm ที่ผสมผสาน Merge Sort กับ Insertion Sort เข้าด้วยกัน
- หลักการทำงานของ Timsort คือ
  - แบ่งข้อมูลออกเป็นส่วนย่อยขนาดเล็กที่เรียกว่า “run” (โดยทั่วไปมีขนาดประมาณ 32-64 elements)
  - แล้วใช้ Insertion Sort เรียงลำดับแต่ละ run แยกกันก่อน (เพราะมีประสิทธิภาพสูงมากบนข้อมูลขนาดเล็ก)
  - จากนั้นจึงใช้ Merge Sort รวม run ทั้งหมดเข้าด้วยกันเป็นผลลัพธ์สุดท้าย
- Timsort มีคุณสมบัติที่ดีของทั้งสองวิธี ได้แก่
  - ความซับซ้อน O(n log n) ที่รับประกันได้เสมอ (จากกลไกของ Merge Sort), เป็น stable sort, และมีประสิทธิภาพที่โดดเด่นเป็นพิเศษบนข้อมูลที่เรียงลำดับบางส่วนอยู่แล้ว ซึ่งเป็นลักษณะที่พบได้บ่อยมากในข้อมูลจริง
- ในทางปฏิบัติจริง การเรียกใช้ sorted() หรือ .sort() ของ Python โดยตรงจึงมักเป็นทางเลือกที่ดีที่สุดในบริบทส่วนใหญ่มากกว่าการ implement sorting algorithm ขึ้นมาเองจากศูนย์

Code: Merge Sort

Merge Sort

```python
def merge(left: list, right: list) -> list:
    """
    รวม list ที่เรียงแล้วสองชุดให้เป็น list เดียวที่เรียงแล้ว
    Complexity: O(n) โดย n = len(left) + len(right)
    Stable: ใช้ <= เพื่อรักษาลำดับเดิมของ element ที่เท่ากัน
    """
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:       # <= รักษา stability (ดูหัวข้อที่ 8)
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])           # เพิ่มส่วนที่เหลือของ left
    result.extend(right[j:])          # เพิ่มส่วนที่เหลือของ right
    return result
```

```python
def merge_sort(arr: list) -> list:
    """
    Merge Sort: Divide and Conquer
    Best/Average/Worst: O(n log n) | Space: O(n) | Stable: Yes
    """
    if len(arr) <= 1:                 # base case
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])      # Divide + Conquer (ซ้าย)
    right = merge_sort(arr[mid:])     # Divide + Conquer (ขวา)
    return merge(left, right)         # Combine
```

ทดสอบ:

```python
data = [8, 3, 5, 1, 9, 2, 7, 4]
result = merge_sort(data)
print(f"Input : {data}")
print(f"Output: {result}")
```

Expected Output

```
Input : [8, 3, 5, 1, 9, 2, 7, 4]
Output: [1, 2, 3, 4, 5, 7, 8, 9]
```

Quick Sort (Last-element Pivot)

Quick Sort (Last-element Pivot)

```python
def _partition_last(arr: list, low: int, high: int) -> int:
    """
    Lomuto partition โดยใช้ arr[high] เป็น pivot
    จัดให้ element ที่ <= pivot อยู่ซ้าย, > pivot อยู่ขวา
    Return: ตำแหน่งสุดท้ายของ pivot หลัง partition
    """
    pivot = arr[high]
    i = low - 1                      # i ชี้ไปยังตำแหน่งสุดท้ายของ "zone ที่ <= pivot"
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]    # วาง pivot ในตำแหน่งที่ถูกต้อง
    return i + 1
```

```python
def quick_sort_last(arr: list, low: int = 0, high: int = None) -> list:
    """
    Quick Sort ด้วย last-element pivot
    Average: O(n log n) | Worst: O(n²) บน sorted input | Space: O(log n)
    """
    if high is None:
        arr = arr.copy()
        high = len(arr) - 1
    if low < high:                   # base case: low >= high (0 หรือ 1 element)
        pivot_idx = _partition_last(arr, low, high)
        quick_sort_last(arr, low, pivot_idx - 1)
        quick_sort_last(arr, pivot_idx + 1, high)
    return arr
```

ทดสอบพื้นฐาน:

```python
data = [8, 3, 5, 1, 9, 2, 7, 4]
print(f"Quick Sort (last pivot): {quick_sort_last(data)}")
```

Expected Output

```
Quick Sort (last pivot): [1, 2, 3, 4, 5, 7, 8, 9]
```

Quick Sort (Median-of-Three Pivot)

```python
def _median_of_three(arr: list, low: int, high: int) -> None:
    """
    เรียง arr[low], arr[mid], arr[high] ใน place
    แล้วย้าย median มาที่ arr[high] เพื่อใช้เป็น pivot
    หลังเรียก: arr[low] <= arr[mid] <= arr[high] แล้ว arr[high] คือ pivot
    """
    mid = (low + high) // 2
    if arr[low] > arr[mid]:
        arr[low], arr[mid] = arr[mid], arr[low]
    if arr[low] > arr[high]:
        arr[low], arr[high] = arr[high], arr[low]
    if arr[mid] > arr[high]:
        arr[mid], arr[high] = arr[high], arr[mid]
    # ตอนนี้ arr[low] <= arr[mid] <= arr[high]
    # median อยู่ที่ arr[mid] → ย้ายมาที่ arr[high] เพื่อใช้เป็น pivot
    arr[mid], arr[high] = arr[high], arr[mid]
```

```python
def quick_sort_median3(arr: list, low: int = 0, high: int = None) -> list:
    """
    Quick Sort ด้วย median-of-three pivot
    หลีกเลี่ยง worst case บน sorted และ reverse-sorted input
    """
    if high is None:
        arr = arr.copy()
        high = len(arr) - 1
    if low < high:
        _median_of_three(arr, low, high)              # เตรียม pivot ที่ arr[high]
        pivot_idx = _partition_last(arr, low, high)   # partition รอบ pivot
        quick_sort_median3(arr, low, pivot_idx - 1)
        quick_sort_median3(arr, pivot_idx + 1, high)
    return arr
```

- สังเกตว่า quick_sort_median3 ใช้ _partition_last ฟังก์ชันเดียวกับที่ quick_sort_last ใช้ทุกประการ
- ความแตกต่างเพียงจุดเดียว คือ การเรียก _median_of_three() ก่อนเรียก partition เพื่อสลับตำแหน่ง median เข้ามาไว้ที่ arr[high] ก่อนที่ partition จะเริ่มทำงาน
  - partition ไม่จำเป็นต้องรู้เลยว่า pivot ที่ตนได้รับมาถูกเลือกด้วยกลยุทธ์ใด เพราะมันมองเห็นเพียงว่า pivot คือ arr[high] เสมอ
- นี่ตัวอย่างที่ดีในการออกแบบซอฟต์แวร์ — แยกหน้าที่ “เลือก pivot” ออกจากหน้าที่ “partition รอบ pivot” อย่างชัดเจน
  - ทำให้สามารถเปลี่ยนกลยุทธ์การเลือก pivot ได้อย่างอิสระโดยไม่ต้องแก้ไข logic ของ partition

Benchmark เปรียบเทียบสอง Pivot Strategy

```
import time

n = 800   # ใช้ n ที่ไม่เกิน recursion limit เพื่อให้ last-pivot ทำงานสำเร็จได้จริง
sorted_data = list(range(n))

start = time.perf_counter()
quick_sort_last(sorted_data.copy())
t_last = time.perf_counter() - start

start = time.perf_counter()
quick_sort_median3(sorted_data.copy())
t_med3 = time.perf_counter() - start

print(f"Sorted input (n={n}):")
print(f"  Last pivot    : {t_last:.4f} วินาที")
print(f"  Median-of-3   : {t_med3:.4f} วินาที")
print(f"  เร็วขึ้น: {t_last / t_med3:.1f}x")
```

Sorted input (n=800):
  Last pivot   : 0.0188 วินาที
  Median-of-3   : 0.0005 วินาที
  เร็วขึ้น: 37.4x